# 7
# Number Theory

Number theory is perhaps the most interesting and beautiful area of mathematics. Euclid's proof that there are an infinite number of primes remains just as clear and elegant today as it was more than two thousand years ago. Innocent-looking questions like whether the equation $a^n + b^n = c^n$ has solutions for integer values of $a$, $b$, $c$, and $n > 2$ often turn out not to be so innocent. Indeed, this is the statement of Fermat's last theorem!

Number theory is great training in formal, rigorous reasoning, because number-theoretic proofs are clear and decisive. Studying the integers is interesting because they are such concrete and important objects. Discovering new properties of the integers is discovering something exciting about the natural world.

Computers have long been used in number theoretic research. Performing interesting number-theoretic computations on large integers requires great efficiency. Fortunately there are many clever algorithms to help us out.

## 7.1 Prime Numbers

A *prime number* is an integer $p > 1$ which is only divisible by 1 and itself. Said another way, if $p$ is a prime number, then $p = a \cdot b$ for integers $a \leq b$ implies that $a = 1$ and $b = p$. The first ten prime numbers are 2, 3, 5, 7, 11, 13, 17, 19, 23, and 27.

Prime numbers are important because of the *fundamental theorem of arithmetic*. Despite the impressive title, all it states is that every integer can be expressed in only one way as the product of primes. For example, 105 is uniquely expressed as $3 \times 5 \times 7$, while 32 is uniquely expressed as $2 \times 2 \times 2 \times 2 \times 2$. This unique set of numbers multiplying

to $n$ is called the *prime factorization* of $n$. Order doesn't matter in a prime factorization, so we can canonically list the numbers in sorted order. But multiplicity does; it is what distinguishes the prime factorization of 4 from 8.

We say a prime number $p$ is a *factor* of $x$ if it appears in its prime factorization. Any number which is not prime is said to be *composite*.

### *7.1.1 Finding Primes*

The easiest way to test if a given number $x$ is prime uses repeated division. Start from the smallest candidate divisor, and then try all possible divisors up from there. Since 2 is the only even prime, once we verify that $x$ isn't even we only need try the odd numbers as candidate factors. Further, we can bless $n$ as prime the instant we have shown that it has no non-trivial prime factors below $\sqrt{n}$. Why? Suppose not – i.e., $x$ is composite but has a smallest non-trivial prime factor $p$ which is greater than $\sqrt{n}$. Then $x/p$ must also divide $x$, and must be larger than $p$, or else we would have seen it earlier. But the product of two numbers greater than $\sqrt{n}$ must be larger than $n$, a contradiction.

Computing the prime factorization involves not only finding the first prime factor, but stripping off all occurrences of this factor and recurring on the remaining product:

```
prime_factorization(long x)
{
        long i;                 /* counter */
        long c;                 /* remaining product to factor */

        c = x;
        while ((c % 2) == 0) {
                printf("%ld\n",2);
                c = c / 2;
        }

        i = 3;
        while (i <= (sqrt(c)+1)) {
                if ((c % i) == 0) {
                        printf("%ld\n",i);
                        c = c / i;
                }
                else
                        i = i + 2;
        }

        if (c > 1) printf("%ld\n",c);
}
```

Testing the terminating condition $i > \sqrt{c}$ is somewhat problematic, because `sqrt()` is a numerical function with imperfect precision. Just to be safe, we let $i$ run an extra

iteration. Another approach would be to avoid floating point computation altogether and terminate when `i*i > c`. However, the multiplication might cause overflow when working on very large integers. Multiplication can be avoided by observing that $(i+1)^2 = i^2 + 2i + 1$, so adding $i + i + 1$ to $i^2$ yields $(i + 1)^2$.

For higher performance, we could move the `sqrt(c)` computation outside the main loop and only update it when `c` changes value. However, this program responds instantly on my computer for the prime 2,147,483,647. There exist fascinating randomized algorithms which are more efficient for testing primality on very large integers, but these are not something for us to worry about at this scale – except as the source of interesting contest problems themselves.

### *7.1.2 Counting Primes*

How many primes are there? It makes sense that primes become rarer and rarer as we consider larger and larger numbers, but do they ever vanish? The answer is no, as shown by Euclid's proof that there are an infinite number of primes. It uses an elegant *proof by contradiction*. Knowing this proof is not strictly necessary to compete in programming contests, but it is one sign of being an educated person. Thus there is no shame in reviewing it here.

Let us assume the converse, that there are only a finite number of primes, $p_1, p_2, \ldots, p_n$. Let $m = 1 + \prod_{i=1}^{n} p_i$, i.e., the product of all of these primes, plus one. Since this is bigger than any of the primes on our list, $m$ must be composite. Therefore some prime must divide it.

But which prime? We know that $m$ is not divisible by $p_1$, because it leaves a remainder of 1. Further, $m$ is not divisible by $p_2$, because it also leaves a remainder of 1. In fact, $m$ leaves a remainder of 1 when divided by any prime $p_i$, for $1 \leq i \leq n$. Thus $p_1, p_2, \ldots, p_n$ cannot be the complete list of primes, because if so $m$ must also be prime.

Since this contradicts the assumption it means there cannot exist such a complete list of primes; therefore the number of primes must be infinite! QED! [1]

Not only are there an infinite number of primes, but in fact the primes are relatively common. There are roughly $x/\ln x$ primes less than or equal to $x$, or put another way, roughly one out of every $\ln x$ numbers is prime.

## 7.2 Divisibility

Number theory is the study of integer divisibility. We say $b$ *divides* $a$ (denoted $b|a$) if $a = bk$ for some integer $k$. Equivalently, we say that $b$ is a *divisor of* $a$ or $a$ is a *multiple* of $b$ if $b|a$.

[1] Now a little puzzle to test your understanding of the proof. Suppose we take the first $n$ primes, multiply them together, and add one. Does this number have to be prime? Give a proof or a counterexample.

As a consequence of this definition, the smallest natural divisor of every non-zero integer is 1. Why? It should be clear that there is in general no integer $k$ such that $a = 0 \cdot k$.

How do we find all divisors of a given integer? From the prime number theorem, we know that $x$ is uniquely represented by the product of its prime factors. Every divisor is the product of some subset of these prime factors. Such subsets can be constructed using backtracking techniques as discussed in Chapter 8, but we must be careful about duplicate prime factors. For example, the prime factorization of 12 has three terms (2, 2, and 3) but 12 has only 6 divisors (1, 2, 3, 4, 6, 12).

### *7.2.1 Greatest Common Divisor*

Since 1 divides every integer, the least common divisor of every pair of integers $a$, $b$ is 1. Far more interesting is the *greatest common divisor*, or *gcd*, the *largest* divisor shared by a given pair of integers. Consider a fraction $x/y$, say, 24/36. The reduced form of this fraction comes after we divide both the numerator and denominator by $gcd(x, y)$, in this case 12. We say two integers are *relatively prime* if their greatest common divisor is 1.

Euclid's algorithm for finding the greatest common divisor of two integers has been called history's first interesting algorithm. The naive way to compute gcd would be to test all divisors of the first integer explicitly on the second, or perhaps to find the prime factorization of both integers and take the product of all factors in common. But both approaches involve computationally intensive operations.

Euclid's algorithm rests on two observations. First,

> If $b|a$, then $gcd(a, b) = b$.

This should be pretty clear. If $b$ divides $a$, then $a = bk$ for some integer $k$, and thus $gcd(bk, b) = b$. Second,

> If $a = bt + r$ for integers $t$ and $r$, then $gcd(a, b) = gcd(b, r)$.

Why? By definition, $gcd(a, b) = gcd(bt + r, b)$. Any common divisor of $a$ and $b$ must rest totally with $r$, because $bt$ clearly must be divisible by any divisor of $b$.

Euclid's algorithm is recursive, repeated replacing the bigger integer by its remainder mod the smaller integer. This typically cuts one of the arguments down by about half, and so after a logarithmic number of iterations gets down to the base case. Consider the following example. Let $a = 34398$ and $b = 2132$.

$$\begin{aligned} gcd(34398, 2132) &= gcd(34398 \bmod 2132, 2132) = gcd(2132, 286) \\ gcd(2132, 286) &= gcd(2132 \bmod 286, 286) = gcd(286, 130) \\ gcd(286, 130) &= gcd(286 \bmod 130, 130) = gcd(130, 26) \\ gcd(130, 26) &= gcd(130 \bmod 26, 26) = gcd(26, 0) \end{aligned}$$

Therefore, $gcd(34398, 2132) = 26$.

However, Euclid's algorithm can give us more than just the $gcd(a, b)$. It can also find integers $x$ and $y$ such that

$$a \cdot x + b \cdot y = gcd(a, b)$$

which will prove quite useful in solving linear congruences. We know that $gcd(a, b) = gcd(b, a')$, where $a' = a - b\lfloor a/b \rfloor$. Further, assume we know integers $x'$ and $y'$ such that

$$b \cdot x' + a' \cdot y' = gcd(a, b)$$

by recursion. Substituting our formula for $a'$ into the above expression gives us

$$b \cdot x' + (a - b\lfloor a/b \rfloor) \cdot y' = gcd(a, b)$$

and rearranging the terms will give us our desired $x$ and $y$. We need a basis case to complete our algorithm, but that is easy since $a \cdot 1 + 0 \cdot 0 = gcd(a, 0)$.

For the previous example, we get that $34398 \times 15 + 2132 \times -242 = 26$. An implementation of this algorithm follows below:

```
/*      Find the gcd(p,q) and x,y such that p*x + q*y = gcd(p,q)      */

long gcd(long p, long q, long *x, long *y)
{
        long x1,y1;                     /* previous coefficients */
        long g;                         /* value of gcd(p,q) */

        if (q > p) return(gcd(q,p,y,x));

        if (q == 0) {
                *x = 1;
                *y = 0;
                return(p);
        }

        g = gcd(q, p%q, &x1, &y1);

        *x = y1;
        *y = (x1 - floor(p/q)*y1);

        return(g);
}
```

### *7.2.2 Least Common Multiple*

Another useful function on two integers is the *least common multiple* (lcm), the *smallest* integer which is divided by both of a given pair of integers. For example, the least common multiple of 24 and 36 is 72.

Least common multiple arises when we want to compute the simultaneous periodicity of two distinct periodic events. When is the next year (after 2000) that the presidential

election (which happens every 4 years) will coincide with census (which happens every 10 years)? The events coincide every twenty years, because $lcm(4, 10) = 20$.

It is self-evident that $lcm(x, y) \geq \max(x, y)$. Similarly, since $x \cdot y$ is a multiple of both $x$ and $y$, $lcm(x, y) \leq xy$. The only way that there can be a smaller common multiple is if there is some non-trivial factor shared between $x$ and $y$.

This observation, coupled with Euclid's algorithm, gives an efficient way to compute least common multiple, namely, $lcm(x, y) = xy/gcd(x, y)$. A slicker algorithm appears in [Dij76], which avoids the multiplication and hence possibility of overflow.

## 7.3 Modular Arithmetic

In Chapter 5, we reviewed the basic arithmetic algorithms for integers, such as addition and multiplication. We are not always interested in the full answers, however. Sometimes the remainder suffices for our purposes. For example, suppose your birthday this year falls on a Wednesday. What day of the week will it it will fall on next year? All you need to know is the remainder of the number of days between now and then (either 365 or 366) when dividing by the 7 days of the week. Thus it will fall on Wednesday plus one (365 mod 7) or two (366 mod 7) days, i.e., Thursday or Friday depending upon whether it is affected by a leap year.

The key to such efficient computations is *modular arithmetic*. Of course, we can in principle explicitly compute the entire number and then find the remainder. But for large enough integers, it can be much easier to just work with remainders via modular arithmetic.

The number we are dividing by is called the *modulus*, and the remainder left over is called the *residue*. The key to efficient modular arithmetic is understanding how the basic operations of addition, subtraction, and multiplication work over a given modulus:

- *Addition* — What is $(x + y) \bmod n$? We can simplify this to

$$((x \bmod n) + (y \bmod n)) \bmod n$$

to avoid adding big numbers. How much small change will I have if given \$123.45 by my mother and \$94.67 by my father?

$$(12,345 \bmod 100) + (9,467 \bmod 100) = (45 + 67) \bmod 100 = 12 \bmod 100$$

- *Subtraction* — Subtraction is just addition with negative values. How much small change will I have after spending \$52.53?

$$(12 \bmod 100) - (53 \bmod 100) = -41 \bmod 100 = 59 \bmod 100$$

Notice how we can convert a negative number mod $n$ to a positive number by adding a multiple of $n$ to it. Further, this answer makes sense in this change example. It is usually best to keep the residue between 0 and $n - 1$ to ensure we are working with the smallest-magnitude numbers possible.

- *Multiplication* — Since multiplication is just repeated addition,

$$xy \bmod n = (x \bmod n)(y \bmod n) \bmod n$$

How much change will you have if you earn $17.28 per hour for 2,143 hours?

$$(1,728 \times 2,143) \bmod 100 = (28 \bmod 100) \times (43 \bmod 100) = 4 \bmod 100$$

Further, since exponentiation is just repeated multiplication,

$$x^y \bmod n = (x \bmod n)^y \bmod n$$

Since exponentiation is the quickest way to produce really large integers, this is where modular arithmetic really proves its worth.

- *Division* — Division proves considerably more complicated to deal with, and will be discussed in Section 7.4.

Modular arithmetic has many interesting applications, including:

- *Finding the Last Digit* — What is the last digit of $2^{100}$? Sure we can use infinite precision arithmetic and look at the last digit, but why? We can do this computation by hand. What we really want to know is what $2^{100} \bmod 10$ is. By doing repeated squaring, and taking the remainder $\bmod 10$ at each step we make progress very quickly:

$$\begin{aligned}
2^3 \bmod 10 &= 8 \\
2^6 \bmod 10 &= 8 \times 8 \bmod 10 \to 4 \\
2^{12} \bmod 10 &= 4 \times 4 \bmod 10 \to 6 \\
2^{24} \bmod 10 &= 6 \times 6 \bmod 10 \to 6 \\
2^{48} \bmod 10 &= 6 \times 6 \bmod 10 \to 6 \\
2^{96} \bmod 10 &= 6 \times 6 \bmod 10 \to 6 \\
2^{100} \bmod 10 &= 2^{96} \times 2^3 \times 2^1 \bmod 10 \to 6
\end{aligned}$$

- *RSA Encryption Algorithm* — A classic application of modular arithmetic on large integers arises in public-key cryptography, namely, the RSA algorithm. Here, our message is encrypted by coding it as an integer $m$, raising it to a power $k$, where $k$ is the so-called public-key or encryption key, and taking the results mod $n$. Since $m$, $n$, and $k$ are all huge integers, computing $m^k \bmod n$ efficiently requires the tools we developed above.

- *Calendrical Calculations* — As demonstrated with the birthday example, computing the day of the week a certain number of days from today, or the time a certain number of seconds from now, are both applications of modular arithmetic.

## 7.4 Congruences

*Congruences* are an alternate notation for representing modular arithmetic. We say that $a \equiv b(\text{mod } m)$ if $m|(a-b)$. By definition, if $a \bmod m$ is $b$, then $a \equiv b(\text{mod } m)$.

Congruences are an alternate notation for modular arithmetic, not an inherently different idea. Yet the notation is important. It gets us thinking about the *set* of integers with a given remainder $n$, and gives us equations for representing them. Suppose that $x$ is a variable. What integers $x$ satisfy the congruence $x \equiv 3(\text{mod } 9)$?

For such a simple congruence, the answer is easy. Clearly $x = 3$ must be a solution. Further, adding or deleting the modulus (9 in this instance) gives another solution. The set of solutions is all integers of the form $9y + 3$, where $y$ is any integer.

What about complicated congruences, such as $2x \equiv 3(\text{mod } 9)$ and $2x \equiv 3(\text{mod } 4)$? Trial and error should convince you that exactly the integers of the form $9y + 6$ satisfy the first example, while the second has no solutions at all.

There are two important problems on congruences, namely, performing arithmetic operations on them, and solving them. These are discussed in the sections below.

### *7.4.1 Operations on Congruences*

Congruences support addition, subtraction, and multiplication, as well as a limited form of division – provided they share the same modulus:

- *Addition and Subtraction* — Suppose $a \equiv b(\text{mod } n)$ and $c \equiv d(\text{mod } n)$. Then $a + c \equiv b + d(\text{mod } n)$. For example, suppose I know that $4x \equiv 7(\text{mod } 9)$ and $3x \equiv 3(\text{mod } 9)$. Then

$$4x - 3x \equiv 7 - 3(\text{mod } 9) \rightarrow x \equiv 4(\text{mod } 9)$$

- *Multiplication* — It is apparent that $a \equiv b(\text{mod } n)$ implies that $a \cdot d \equiv b \cdot d(\text{mod } n)$ by adding the reduced congruence to itself $d$ times. In fact, general multiplication also holds, i.e., $a \equiv b(\text{mod } n)$ and $c \equiv d(\text{mod } n)$ implies $ac \equiv bd(\text{mod } n)$.

- *Division* — However, we cannot cavalierly cancel common factors from congruences. Note that $6 \cdot 2 \equiv 6 \cdot 1(\text{mod } 3)$, but clearly $2 \not\equiv 1(\text{mod } 3)$. To see what the problem is, note that we can redefine division as multiplication by an inverse, so $a/b$ is equivalent to $ab^{-1}$. Thus we can compute $a/b(\text{mod } n)$ if we can find the inverse $b^{-1}$ such that $bb^{-1} \equiv 1(\text{mod } n)$. This inverse does not always exist – try to find a solution to $2x \equiv 1(\text{mod } 4)$.

  We *can* simplify a congruence $ad \equiv bd(\text{mod } dn)$ to $a \equiv b(\text{mod } n)$, so we can divide all three terms by a mutually common factor if one exists. Thus $170 \equiv 30(\text{mod } 140)$ implies that $17 \equiv 3(\text{mod } 14)$. However, the congruence $a \equiv b(\text{mod } n)$ must be false (i.e., has no solution) if $gcd(a, n)$ does not divide $b$.

### 7.4.2 Solving Linear Congruences

A linear congruence is an equation of the form $ax \equiv b(\text{mod } n)$. Solving this equation means identifying which values of $x$ satisfy it.

Not all such equations have solutions. We have seen integers which do not have multiplicative inverses over a given modulus, meaning that $ax \equiv 1(\text{mod } n)$ has no solution. In fact, $ax \equiv 1(\text{mod } n)$ has a solution if and only if the modulus and the multiplier are relatively prime, i.e., $gcd(a, n) = 1$. We may use Euclid's algorithm to find this inverse through the solution to $a \cdot x' + n \cdot y' = gcd(a, n) = 1$. Thus

$$ax \equiv 1(\text{mod } n) \rightarrow ax \equiv a \cdot x' + n \cdot y'(\text{mod } n)$$

Clearly $n \cdot y' \equiv 0(\text{mod } n)$, so in fact this inverse is simply the $x'$ from Euclid's algorithm.

In general, there are three cases, depending on the relationship between $a$, $b$, and $n$:

- $\gcd(a, b, n) > 1$ — Then we can divide all three terms by this divisor to get an equivalent congruence. This gives us a single solution mod the new base, or equivalently $\gcd(a, b, n)$ solutions $(\text{mod } n)$.
- *$gcd(a, n)$ does not divide $b$* — Then, as described above, the congruence can have no solution.
- *$gcd(a, n) = 1$* — Then there is one solution $(\text{mod } n)$. Further, $x = a^{-1}b$ works, since $aa^{-1}b \equiv b(\text{mod } n)$. As shown above, this inverse exists and can be found using Euclid's algorithm.

The *Chinese remainder theorem* gives us a tool for working with systems of congruences over different moduli. Suppose there is exists an integer $x$ such that $x \equiv a_1(\text{mod } m_1)$ and $x \equiv a_2(\text{mod } m_2)$. Then $x$ is uniquely determined $(\text{mod } m_1 m_2)$ if $m_1$ and $m_2$ are relatively prime.

To find this $x$, and thus solve the system of two congruences, we begin by solving the linear congruences $m_2 b_1 \equiv 1(\text{mod } m_1)$ and $m_1 b_1 \equiv 1(\text{mod } m_2)$ to find $b_1$ and $b_2$ respectively. Then it can be readily verified that

$$x = a_1 b_1 m_2 + a_2 b_2 m_1$$

is a solution to both of the original congruences. Further, the theorem readily extends to systems of an arbitrary number of congruences whose moduli are all pairwise relatively prime.

### 7.4.3 Diophantine Equations

*Diophantine equations* are formulae in which the variables are restricted to integers. For example, Fermat's last theorem concerned answers to the equation $a^n + b^n = c^n$. Solving such an equation for real numbers is no big deal. It is only if all variables are restricted to integers that the problem becomes difficult.

Diophantine equations are difficult to work with because division is not a routine operation with integer formulae. However, there are certain classes of Diophantine equations which are known to be solvable and these tend to arise frequently.

The most important class are linear Diophantine equations of the form $ax - ny = b$, where $x$ and $y$ are the integer variables and $a$, $b$, and $n$ are integer constants. These are readily shown to be equivalent to the solving the congruence $ax \equiv b (\mathrm{mod}\ n)$ and hence can be solved using the techniques of the previous section.

More advanced Diophantine analysis is beyond the scope of this book, but we refer the reader to standard references in number theory such as Niven and Zuckerman [ZMNN91] and Hardy and Wright [HW79] for more on this fascinating subject.

## 7.5 Number Theoretic Libraries

The Java `BigInteger` class (`java.math.BigInteger`) includes a variety of useful number-theoretic functions. Most important, of course, is the basic support for arithmetic operations on arbitrary-precision integers as discussed in Chapter 5. But there are also several functions of purely number-theoretic interest:

- *Greatest Common Divisor* — `BigInteger gcd(BigInteger val)` returns the BigInteger whose value is the gcd of abs(this) and abs(val).
- *Modular Exponentiation* — `BigInteger modPow(BigInteger exp, BigInteger m)` returns a BigInteger whose value is $this^{exp}$ mod $m$.
- *Modular Inverse* — `BigInteger modInverse(BigInteger m)` returns a BigInteger whose value is $this^{-1}(\mathrm{mod}\ m)$, i.e. solves the congruence $y \cdot this \equiv 1(\mathrm{mod}\ m)$ by returning an appropriate integer $y$ if it exists.
- *Primality Testing* — `public boolean isProbablePrime(int certainty)` uses a randomized primality test to return true if this BigInteger is probably prime and false if it's definitely composite. If the call returns true, the probability of primality is $\geq 1 - 1/2^{certainty}$.

## 7.6 Problems

### *7.6.1 Light, More Light*

**PC/UVa IDs:** 110701/10110, **Popularity:** A, **Success rate:** average **Level:** 1

There is man named Mabu who switches on-off the lights along a corridor at our university. Every bulb has its own toggle switch that changes the state of the light. If the light is off, pressing the switch turns it on. Pressing it again will turn it off. Initially each bulb is off.

He does a peculiar thing. If there are $n$ bulbs in a corridor, he walks along the corridor back and forth $n$ times. On the $i$th walk, he toggles only the switches whose position is divisible by $i$. He does not press any switch when coming back to his initial position. The $i$th walk is defined as going down the corridor (doing his peculiar thing) and coming back again. Determine the final state of the last bulb. Is it on or off?

#### *Input*

The input will be an integer indicating the $n$th bulb in a corridor, which is less than or equal to $2^{32} - 1$. A zero indicates the end of input and should not be processed.

#### *Output*

Output "`yes`" or "`no`" to indicate if the light is on, with each case appearing on its own line.

#### *Sample Input*

```
3
6241
8191
0
```

#### *Sample Output*

```
no
yes
no
```

### 7.6.2 Carmichael Numbers

**PC/UVa IDs:** 110702/10006, **Popularity:** A, **Success rate:** average **Level:** 2

Certain cryptographic algorithms make use of big prime numbers. However, checking whether a big number is prime is not so easy.

Randomized primality tests exist that offer a high degree of confidence of accurate determination at low cost, such as the Fermat test. Let $a$ be a random number between 2 and $n-1$, where $n$ is the number whose primality we are testing. Then, $n$ is *probably* prime if the following equation holds:

$$a^n \bmod n = a$$

If a number passes the Fermat test several times, then it is prime with a high probability.

Unfortunately, there is bad news. Certain composite numbers (non-primes) still pass the Fermat test with every number smaller than themselves. These numbers are called Carmichael numbers.

Write a program to test whether a given integer is a Carmichael number.

#### *Input*

The input will consist of a series of lines, each containing a small positive number $n$ ($2 < n < 65,000$). A number $n = 0$ will mark the end of the input, and must not be processed.

#### *Output*

For each number in the input, print whether it is a Carmichael number or not as shown in the sample output.

| *Sample Input* | *Sample Output* |
|---|---|
| 1729 | The number 1729 is a Carmichael number. |
| 17 | 17 is normal. |
| 561 | The number 561 is a Carmichael number. |
| 1109 | 1109 is normal. |
| 431 | 431 is normal. |
| 0 | |

### *7.6.3 Euclid Problem*

**PC/UVa IDs:** 110703/10104, **Popularity:** A, **Success rate:** average **Level:** 1

From Euclid, it is known that for any positive integers $A$ and $B$ there exist such integers $X$ and $Y$ that $AX + BY = D$, where $D$ is the greatest common divisor of $A$ and $B$. The problem is to find the corresponding $X$, $Y$, and $D$ for a given $A$ and $B$.

#### *Input*

The input will consist of a set of lines with the integer numbers $A$ and $B$, separated with space ($A, B < 1,000,000,001$).

#### *Output*

For each input line the output line should consist of three integers $X$, $Y$, and $D$, separated with space. If there are several such $X$ and $Y$, you should output that pair for which $X \leq Y$ and $|X| + |Y|$ is minimal.

#### *Sample Input*

```
4 6
17 17
```

#### *Sample Output*

```
-1 1 2
0 1 17
```

### *7.6.4 Factovisors*

**PC/UVa IDs:** 110704/10139, **Popularity:** A, **Success rate:** average **Level:** 2

The factorial function, $n!$ is defined as follows for all non-negative integers $n$:

$$\begin{aligned} 0! &= 1 \\ n! &= n \times (n-1)! \qquad (n > 0) \end{aligned}$$

We say that $a$ divides $b$ if there exists an integer $k$ such that

$$k \times a = b$$

#### *Input*

The input to your program consists of several lines, each containing two non-negative integers, $n$ and $m$, both less than $2^{31}$.

#### *Output*

For each input line, output a line stating whether or not $m$ divides $n!$, in the format shown below.

#### *Sample Input*

```
6 9
6 27
20 10000
20 100000
1000 1009
```

#### *Sample Output*

```
9 divides 6!
27 does not divide 6!
10000 divides 20!
100000 does not divide 20!
1009 does not divide 1000!
```

### *7.6.5 Summation of Four Primes*

**PC/UVa IDs:** 110705/10168, **Popularity:** A, **Success rate:** average **Level:** 2

Waring's prime number conjecture states that every odd integer is either prime or the sum of three primes. Goldbach's conjecture is that every even integer is the sum of two primes. Both problems have been open for over 200 years.

In this problem you have a slightly less demanding task. Find a way to express a given integer as the sum of exactly four primes.

#### *Input*

Each input case consists of one integer $n$ ($n \leq 10000000$) on its own line. Input is terminated by end of file.

#### *Output*

For each input case $n$, print one line of output containing four prime numbers which sum up to $n$. If the number cannot be expressed as a summation of four prime numbers print the line "`Impossible.`" in a single line. There can be multiple solutions. Any good solution will be accepted.

#### *Sample Input*

```
24
36
46
```

#### *Sample Output*

```
3 11 3 7
3 7 13 13
11 11 17 7
```

### *7.6.6 Smith Numbers*

**PC/UVa IDs:** 110706/10042, **Popularity:** B, **Success rate:** average **Level:** 1

While skimming his phone directory in 1982, mathematician Albert Wilansky noticed that the telephone number of his brother-in-law H. Smith had the following peculiar property: The sum of the digits of that number was equal to the sum of the digits of the prime factors of that number. Got it? Smith's telephone number was 493-7775. This number can be written as the product of its prime factors in the following way:

$$4937775 = 3 \cdot 5 \cdot 5 \cdot 65837$$

The sum of all digits of the telephone number is $4 + 9 + 3 + 7 + 7 + 7 + 5 = 42$, and the sum of the digits of its prime factors is equally $3 + 5 + 5 + 6 + 5 + 8 + 3 + 7 = 42$. Wilansky named this type of number after his brother-in-law: the Smith numbers.

As this property is true for every prime number, Wilansky excluded them from the definition. Other Smith numbers include 6,036 and 9,985.

Wilansky was not able to find a Smith number which was larger than the telephone number of his brother-in-law. Can you help him out?

#### *Input*

The input consists of several test cases, the number of which you are given in the first line of the input. Each test case consists of one line containing a single positive integer smaller than $10^9$.

#### *Output*

For every input value $n$, compute the smallest Smith number which is larger than $n$ and print it on a single line. You can assume that such a number exists.

#### *Sample Input*

```
1
4937774
```

#### *Sample Output*

```
4937775
```

### *7.6.7 Marbles*

**PC/UVa IDs:** 110707/10090, **Popularity:** B, **Success rate:** low **Level:** 1

I collect marbles (colorful small glass balls) and want to buy boxes to store them. The boxes come in two types:

*Type* 1**:** each such box costs $c_1$ dollars and can hold exactly $n_1$ marbles

*Type* 2**:** each such box costs $c_2$ dollars and can hold exactly $n_2$ marbles

I want each box to be filled to its capacity, and also to minimize the total cost of buying them. Help me find the best way to distribute my marbles among the boxes.

#### *Input*

The input file may contain multiple test cases. Each test case begins with a line containing the integer $n$ ($1 \leq n \leq$ 2,000,000,000). The second line contains $c_1$ and $n_1$, and the third line contains $c_2$ and $n_2$. Here, $c_1$, $c_2$, $n_1$, and $n_2$ are all positive integers having values smaller than 2,000,000,000.

A test case containing a zero for the number of marbles terminates the input.

#### *Output*

For each test case in the input print a line containing the minimum cost solution (two nonnegative integers $m_1$ and $m_2$, where $m_i$ = number of type $i$ boxes required if one exists. Otherwise print "`failed`".

If a solution exists, you may assume that it is unique.

#### *Sample Input*

```
43
1 3
2 4
40
5 9
5 12
0
```

#### *Sample Output*

```
13 1
failed
```

### 7.6.8 Repackaging

**PC/UVa IDs:** 110708/10089, **Popularity:** C, **Success rate:** low **Level:** 2

Coffee cups of three different sizes (size 1, size 2, and size 3) are manufactured by the Association of Cup Makers (ACM) and are sold in various packages. Each type of package is identified by three positive integers $(S_1, S_2, S_3)$, where $S_i$ $(1 \leq i \leq 3)$ denotes the number of size $i$ cups included in the package. Unfortunately, there is no package such that $S_1 = S_2 = S_3$.

Market research has discovered there is great demand for packages containing equal numbers of cups of all three sizes. To exploit this opportunity, ACM has decided to unpack the cups from some of the packages in its unlimited stock of unsold products and repack them as packages having equal number of cups of all three sizes. For example, suppose ACM has the following packages in its stock: $(1, 2, 3)$, $(1, 11, 5)$, $(9, 4, 3)$, and $(2, 3, 2)$. Then we can unpack three $(1, 2, 3)$ packages, one $(9, 4, 3)$ package, and two $(2, 3, 2)$ packages and repack the cups to produce sixteen $(1, 1, 1)$ packages. One can even produce eight $(2, 2, 2)$ packages or four $(4, 4, 4)$ packages or two $(8, 8, 8)$ packages or one $(16, 16, 16)$ package, etc. Note that all the unpacked cups must be used to produce the new packages; i.e., no unpacked cup is wasted.

ACM has hired you to write a program to decide whether it is possible to produce packages containing an equal number of all three types of cups using all the cups that can be found by unpacking any combination of existing packages in stock.

#### *Input*

The input may contain multiple test cases. Each test case begins with a line containing an integer $N$ $(3 \leq N \leq 1{,}000)$ indicating the number of different types of packages that can be found in the stock. Each of the next $N$ lines contains three positive integers denoting, respectively, the number of size 1, size 2, and size 3 cups in a package. No two packages in a test case will have the same specification.

A test case containing a zero for $N$ in the first line terminates the input.

#### *Output*

For each test case print a line containing "`Yes`" if packages can be produced as desired. Print "`No`" if they cannot be produced.

#### *Sample Input*

```
4
1 2 3
1 11 5
9 4 3
2 3 2
4
```

```
1 3 3
1 11 5
9 4 3
2 3 2
0
```

*Sample Output*

```
Yes
No
```

## 7.7 Hints

7.1 Can we figure out the state of the $n$th bulb without testing all numbers from 1 to $n$?

7.2 How can we compute $a^n \pmod{n}$ efficiently?

7.3 Are we sure the construction in the text gives the minimal such pair?

7.4 Can we test the divisibility without explicitly computing $n!$?

7.7 Can you compute the possible exact solutions independent of cost? Which one of these will be the cheapest?

7.8 Can we solve these Diophantine equations using the techniques discussed in this chapter?

## 7.8 Notes

7.5 The Goldbach and Waring conjectures are almost certainly true, but perhaps because of brute force instead of deep properties of the primes. Do a back-of-the-envelope calculation of the expected number of solutions for each problem, assuming that there are $n/\ln n$ primes less than $n$. Is it promising to hunt further for a counter-example when none has been found before $n = 1{,}000{,}000$?

7.6 Papers on the properties of Smith numbers include [Wil82, McD87].